

Monitoring

Music Podcast Player

1 INTRODUCTION

Another facet of our project is the Logging, located in the backend, and our Metrics Dashboard, which lives in an AWS Cloudwatch Instance. Our logs cover virtually every interaction a user may have with the application, including both errors, action, and response logging. Our logs are primarily focused on tracking user movements and error tracking.

2 LOGGING COVERAGE

- **200** – Any successful response from the server returned to the user
- **400** – Malformed JSON in Request & Arguments Did Not Match Expected Scheme
- **404** – The Requested Resource Could Not Be Found
- **409** – The Field Is Already in Use
- **429** – The Server is Receiving Too Many Requests to Process the Current One
- **502** – The Internal API Response was Invalid & The External API Response was Invalid
- **503** – The External API Used for this response was invalid
- **504** – We Were Unable to Reach the Internal API & We Were Unable to Reach the External API

3 CLOUDWATCH

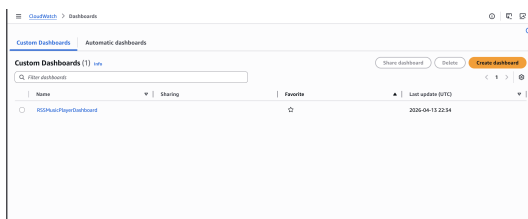


Figure 1: Dashboard Creation

After connecting to the CDK and navigating to cloudwatch, you'll likely be faced with this screen. TheRSSMusicPlayer Dashboard is the initial base dashboard that displays the metrics for all current logs in the application. There are also non-custom dashboards

that display pre-configured metrics for the application as well, but are not configurable. Cloudwatch runs alongside the EC2 Instance and waits for the agent to send the logs to cloudwatch.

4 CREATING NEW LOGS

To create new logs, you must go into the cdk snapshot of the backend, and find the errors.py file. Inside this file there is a class of enums that relate to the errors. To include a request or response error, you update the class with expected error, then create the enumerated type for it.

```
class RequestError(Enum):
    INVALID_FORMAT = auto()
    INVALID_ARGUMENT = auto()
    EXTERNAL_API_TIMEOUT = auto()
    EXTERNAL_API_BAD_RESPONSE = auto()
    EXTERNAL_API_UNAVAILABLE = auto()
    VALUE_NOT_UNIQUE = auto()
    INTERNAL_API_TIMEOUT = auto()
    INTERNAL_API_BAD_RESPONSE = auto()
    NOT_FOUND = auto()
    TOO_MANY_REQUESTS = auto()
```

Figure 2: Error Class

Creating new logs, after updating the class, is as simple as following the structure outlined in the logging config. Each log should contain an error, message, and code to complete it. After the enumerated type is created, it should be added to the class, to be included in the logs that will be returned.

```
RequestError.INVALID_FORMAT: {
    "error": "InvalidFormat",
    "message": "Malformed JSON in request",
    "code": 400,
},
```

Figure 3: Error Format

To update logging structure, navigate to the loggingconfig.py file and locate the log request function. Here, you can update the log structure and add different tags and types to extend them. In here, you can also update the order of keys for the logs. These should only be updated if you decide to change the entire structure of the logs as well.

```
def log_request(
    logger: structlog.BoundLogger,
    level: str,
    event: str,
    message: str,
    user_id: str | None = None,
    route: str = "",
    status_code: int = 200,
    details: dict = {},
    **extra_fields: Any,
) -> None:
    """Log a request event w/ structured fields."""
    log_method = getattr(logger, level)

    log_kwargs = {
        "message": message,
        "userId": user_id or "none",
        "route": route,
        "statusCode": status_code,
    }
```

Figure 4: Log Structure

5 CREATING NEW METRICS

To create new metrics, you have to navigate to the CDK stack file and locate the functions that create the dashboards and metrics. These metrics are updated on a new deployment of the CDK, and a new metric or dashboard will only be able to be seen on the new deployment. To create a new metric, you have to add a new metric filter to the log group. To add the log to the metric, you have to add a filter pattern and metric value. The metric value is how much your log increments when the filter pattern was detected.

```
unauthorized_metric = log_group.add_metric_filter(
    "UnauthorizedFilter",
    metric_name="AuthFailureCount",
    metric_namespace="RSSMusicPlayer",
    filter_pattern=logs.FilterPattern.all_terms("{ $.statusCode = 401 }"),
    metric_value="1",
).metric()
```

Figure 5: Metric Structure

Right below, you will find how to create a new dashboard for the cloudwatch instance as well. To create a new dashboard you have to name and create an update interval for the dashboard. After this, you have reference the metrics you want to include in the dashboard. Once these are referenced and the new dashboard is deployed, you should be able to navigate to your AWS Console and view your newly created metrics and dashboard.

```
dashboard = cloudwatch.Dashboard(
    self,
    "RSSMusicPlayerDashboard",
    dashboard_name="RSSMusicPlayerDashboard",
    default_interval=Duration.minutes(5),
)
```

Figure 6: Dashboard Structure